

AI 사용 보고서

C++ & Qt 프로젝트

ROS2와 Qt를 이용한 Knife Hit 게임 구현

학번: 2026406076

이름: 정창규

제출일: 2026-09-14

목차

1. AI 사용 목적

2. AI를 통해 학습한 내용

2.1 ROS2와 Qt 구조 이해

2.2 원판 회전 및 좌표 계산

2.3 칼 충돌 판정과 랜덤 배치

3. 오류 해결 과정에서의 AI 사용

4. AI 사용 후 느낀 점

1. AI 사용 목적

이번 프로젝트에서는 구현 과정에서 이해되지 않는 개념을 확인하고, 오류가 발생했을 때 원인을 찾기 위해 AI를 사용하였다.

특히 ROS2와 Qt를 함께 사용하는 구조가 익숙하지 않아 QNode, QThread, Topic, Service의 역할을 질문하였다. 또한 원판의 회전 중심을 잡는 방법, 칼의 충돌을 각도 차이로 판정하는 방법, 사과를 원판 둘레에 배치하기 위해 $\sin()$ 과 $\cos()$ 를 사용하는 이유 등을 이해하는 데 활용하였다.

프로그램을 구현하면서 발생한 오류에 대해서도 오류 메시지와 코드의 동작을 함께 확인하였다. 이를 통해 각 코드가 어떤 역할을 하는지 이해하고, 프로젝트의 전체 구조를 정리하는 데 AI를 활용하였다.

2. AI를 통해 학습한 내용

2.1 ROS2와 Qt 구조 이해

프로젝트를 진행하면서 AI를 통해 ROS2와 Qt가 어떤 방식으로 연결되는지 확인하였다. 프로그램은 `game_logic_node`와 `game_gui_node` 두 개의 노드로 구성하였으며, `game_logic_node`에서는 원판 회전, 칼 충돌 판정, 점수와 스테이지 같은 게임 상태를 계산하도록 하였다.

GUI 쪽에서는 QNode와 MainWindow를 나누어 사용하였다. QNode는 ROS2의 Topic과 Service 통신을 담당하고, MainWindow는 Qt 화면과 애니메이션을 처리하도록 구성하였다. 또한 QNode가 QThread를 이용하여 별도의 Thread에서 `rclcpp::spin_some()`을 실행하는 이유도 학습하였다.

이를 통해 Qt의 화면 처리와 ROS2의 통신 처리를 분리하면 서로의 동작을 방해하지 않고 프로그램을 실행할 수 있다는 점을 이해하였다.

QNode가 QThread를 상속받는 이유가 뭐야?

QNode가 QThread를 상속받는 이유는 Qt의 GUI 처리와 ROS2 통신 처리를 서로 분리해서 동시에 실행하기 위해서야.

Qt는 메인 스레드에서 `app.exec()`를 실행하면서 버튼 클릭, 키보드 입력, 화면 갱신, 애니메이션 같은 GUI 이벤트를 계속 처리해. 반면 ROS2는 Topic이나 Service 메시지를 받으려면 `rclcpp::spin()`이나 `rclcpp::spin_some()`을 계속 실행해야 해.

만약 ROS2의 통신 처리를 Qt 메인 스레드에서 같이 실행하면, ROS2의 처리 때문에 GUI가 멈추거나 화면 갱신이 늦어질 수 있어. 그래서 우리 프로젝트에서는 QNode가 QThread를 상속받고, 별도의 스레드에서 ROS2 통신을 처리하도록 구성했어.

예를 들면 QNode의 `run()` 함수에서는 다음과 같이 동작해.

```
</> C++
void QNode::run()
{
    rclcpp::WallRate loop_rate(100);

    while (rclcpp::ok())
    {
        rclcpp::spin_some(node_);
        loop_rate.sleep();
    }
}
```

여기서 `rclcpp::spin_some(node_)`은 현재 들어와 있는 ROS2 메시지를 확인하고 Subscriber나 Service 관련 콜백을 실행하는 역할을 해. `WallRate(100)`은 이 과정을 약 100Hz, 즉 약 10ms 간격으로 반복하도록 해.

구조를 정리하면 다음과 같아.

```
</> 일반 텍스트
메인 스레드
→ MainWindow
→ Qt 화면, 키 입력, 애니메이션 처리

별도 QThread
→ QNode
→ rclcpp::spin_some()
→ ROS2 Topic, Service 통신 처리
```

2.2 원판 회전 및 좌표 계산

원판과 사과의 위치를 계산하는 과정에서 좌표와 각도를 사용하는 방법을 AI를 통해 확인하였다. 원판은 QGraphicsItemGroup의 중심을 기준으로 회전하도록 구성하였고, 원판에 꽂힌 칼과 사과도 같은 그룹에 포함하여 원판과 함께 회전하도록 하였다.

사과를 원판 둘레에 배치할 때에는 $\sin()$ 과 $\cos()$ 를 이용하여 각도에 따른 X, Y 좌표를 계산하였다. 또한 $\text{setPos}()$ 가 이미지의 중심이 아닌 왼쪽 위 위치를 기준으로 하기 때문에, 이미지의 가로와 세로 크기의 절반을 빼서 사과의 중심이 계산된 좌표에 오도록 보정하는 방법을 이해하였다.

원판 회전이 끊겨 보이는 문제를 해결하는 과정에서는 목표 각도와 현재 화면에 표시되는 각도의 차이를 이용하여 조금씩 따라가도록 하는 방법도 학습하였다. 이를 통해 좌표 계산과 화면 표시 방식이 실제 게임 화면에 어떤 영향을 주는지 이해할 수 있었다.

2.3 칼 충돌 판정과 랜덤 배치

칼 충돌 판정에서는 이미지가 직접 겹치는지를 확인하는 대신, 원판을 기준으로 각 칼의 위치를 각도로 저장하고 각도 차이를 비교하는 방식을 사용하였다. AI를 통해 두 각도의 차이를 계산할 때 0도와 360도가 이어지는 점까지 고려해야 한다는 것을 확인하였다.

예를 들어 기존 칼이 134도에 있고 새로 들어오는 칼이 150도라면 두 칼의 차이는 16도가 된다. 이 값이 설정한 충돌 기준인 18도보다 작으면 충돌로 판단하도록 하였다. 이를 통해 좌표를 직접 비교하지 않고도 원판 위에서 칼 사이의 간격을 판단할 수 있다는 점을 이해하였다.

사과와 장애물 칼을 배치할 때에는 랜덤 각도를 생성한 뒤 기존 칼이나 사과와 너무 가까운 위치는 제외하도록 하였다. 이 과정에서 최소 각도 차이를 두어 서로 겹치지 않게 배치하는 방법을 확인하였고, 같은 스테이지를 다시 실행하더라도 위치가 달라지도록 구현할 수 있었다.

칼을 원판에 던졌을 때 기존에 꽂혀 있는 칼이랑 충돌했는지 판단하는 로직은 어떻게 짜는 게 좋을까?

칼 충돌은 원판 위에서 칼의 각도 차이를 비교하는 방식으로 구현하면 좋아.

기존에 꽂혀 있는 칼들의 위치를 각도로 저장해두고, 새 칼이 들어왔을 때 새 칼의 각도와 기존 칼들의 각도를 하나씩 비교하는 거야.

예를 들어 기존 칼이 135°, 새 칼이 150° 라면 두 칼의 차이는 15°야. 충돌 기준을 18°로 잡았다면 충돌로 처리하면 돼.

예시는 이런 식으로 만들 수 있어.

```
</> C++  
double newAngle = /* 새 칼의 원판 기준 각도 */;  
  
for (double oldAngle : knifeAngles)  
{  
    double diff = fabs(newAngle - oldAngle);  
  
    if (diff > 180.0)  
        diff = 360.0 - diff;  
  
    if (diff < 18.0)  
    {  
        gameOver = true;  
        break;  
    }  
}
```

350°와 10°처럼 0°와 360°가 이어지는 경우도 있기 때문에, 각도 차이가 180°보다 크면 $360 - \text{diff}$ 로 다시 계산해주는 게 좋아.

정리하면 새 칼 각도 계산 → 기존 칼들과 각도 차이 비교 → 기준값보다 가까우면 충돌 처리 순서로 구현하면 돼.

사과랑 장애물 칼을 랜덤으로 배치할 때 서로 너무 가까이 겹치지 않게 하려면 로직을 어떻게 짜는 게 좋을까?

📄 ✎

사과랑 장애물 칼을 랜덤으로 배치할 때는 무작정 랜덤 위치를 바로 사용하는 게 아니라, 새로 뽑은 각도가 기존 물체들과 충분히 떨어져 있는지 확인한 뒤 배치하는 방식이 좋아.

예를 들어 새 위치를 하나 뽑았다고 하면:

```
</> C++  
  
double newAngle = randomAngle();  
bool safe = true;  
  
for (double oldAngle : usedAngles)  
{  
    if (angleDifference(newAngle, oldAngle) < 35.0)  
    {  
        safe = false;  
        break;  
    }  
}
```

이렇게 기존에 사용 중인 각도들과 하나씩 비교해서, 최소 간격보다 가까우면 그 위치는 버리고 다시 랜덤 각도를 뽑는 거야.

흐름은 이렇게 보면 돼.

</> 일반 텍스트

📄 ✎

```
랜덤 각도 생성  
↓  
기존 칼/사과와 거리 비교  
↓  
너무 가까움?  
Yes → 다시 랜덤 생성  
No → 해당 위치에 배치
```

장애물 칼과 사과의 최소 간격을 다르게 잡을 수도 있어. 예를  장애물 칼은 42°, 사과는 35° 정도로 두면 서로 너무 붙어서 생성되는 걸 막을 수 있어.

3. 오류 해결 과정에서의 AI 사용

프로젝트를 진행하면서 여러 오류가 발생하였고, 오류 메시지와 코드의 동작을 확인하기 위해 AI를 사용하였다. Qt에서 이미지가 정상적으로 나타나지 않았을 때에는 리소스 파일과 경로 설정을 확인하면서 문제의 원인을 찾았다.

또한 원판 회전이 끊겨 보이는 문제가 발생했을 때에는 ROS2에서 전달받은 각도를 화면에 바로 적용하는 방식 때문에 움직임이 자연스럽지 않을 수 있다는 점을 확인하였다. 이후 현재 화면의 각도가 목표 각도를 조금씩 따라가도록 수정하여 원판이 더 부드럽게 회전하도록 하였다.

ros2_create_qt_pkg 구조로 프로젝트를 다시 구성한 뒤에는 vtable 관련 빌드 오류가 발생하였다.

이 과정에서 Q_OBJECT가 포함된 헤더 파일이 Qt의 AUTOMOC 과정에서 처리되어야 한다는 점을 확인하였고, CMakeLists.txt에 관련 헤더를 추가하여 오류를 해결하였다. 이 과정을 통해 코드 뿐만 아니라 Qt와 ROS2의 빌드 구조도 함께 확인해야 한다는 것을 알게 되었다.

ROS2에서 원판 각도를 받아서 바로 회전시키니까 움직임이 좀 끊겨 보이는데, 부드럽게 회전하게 하려면 어떻게 하는 게 좋을까?

원판 각도를 ROS2에서 받을 때마다 바로 `setRotation()` 으로 적용하면, ROS2 메시지가 들어오는 간격과 Qt 화면이 갱신되는 간격이 다르기 때문에 움직임이 끊겨 보일 수 있어.

이럴 때는 받은 각도를 바로 표시하지 말고, 목표 각도(target)와 현재 화면 각도(visual)를 따로 두고 현재 각도가 목표 각도를 조금씩 따라가게 만들면 돼.

예를 들면 이런 방식이야.

```
</> C++
double diff = targetAngle - visualAngle;

visualAngle += diff * 0.65;

rotatingGroup->setRotation(visualAngle);
```

예를 들어 현재 화면 각도가 `100°` 이고 ROS2에서 받은 목표 각도가 `110°` 라면,

```
</> 일반 텍스트
차이 = 10°

100 + 10 × 0.65
= 106.5°
```

처럼 한 번에 110°로 이동하지 않고 106.5°까지만 이동해. 다음 프레임에서는 남은 차이만큼 다시 이동하면서 점점 110°에 가까워지는 방식이야.

이렇게 하면

ROS2에서 목표 각도 수신 → 현재 각도와 차이 계산 → 차이의 일부만 이동 → 반복

순서로 동작해서 원판이 훨씬 부드럽게 회전해 보여.

`0.65`는 부드러움의 정도를 조절하는 값이야. 값이 크면 목표 를 빠르게 따라가고, 작으면 더 천천히 부드럽게 따라가게 돼.

4. AI 사용 후 느낀 점

이번 프로젝트를 진행하면서 AI를 이용해 이해되지 않는 부분을 바로 확인할 수 있어서 도움이 되었다. 특히 ROS2와 Qt를 함께 사용하는 구조나 좌표 계산, 충돌 판정처럼 처음에는 어렵게 느껴졌던 부분을 코드와 같이 보면서 이해할 수 있었다.

오류가 발생했을 때에도 단순히 코드를 수정하는 것보다 왜 문제가 발생했는지 확인하는 과정이 중요하다고 느꼈다. 원판 회전 문제나 vtable 빌드 오류를 해결하면서 프로그램의 실행 구조와 빌드 과정도 같이 볼 수 있었다.

이번 프로젝트를 통해 AI를 활용하면서 코드의 각 부분이 어떤 역할을 하는지 구분해서 보는 습관이 생겼고, 처음보다 ROS2와 Qt를 연결하는 구조를 더 잘 이해할 수 있었다.